기쁨의교회 홈페이지 개발 규칙 문서 (Development Constitution)

> *이 문서는 기쁨의교회 홈페이지 프로젝트의 모든 개발 작업에 적용되는 기준입니다. 새로운 기능을 추가하거나 기존 코드를 수정할 때 반드시 이 문서를 먼저 확인하고 따라야 합니다. 작성일: 2026년 4월 | 관리: 기쁨의교회 홈페이지 개발팀*

목차

1. 프로젝트 철학
2. 기술 스택 규칙
3. 폴더 및 파일 구조 규칙
4. 코드 작성 규칙
5. 데이터베이스 규칙
6. API 설계 규칙
7. 보안 규칙
8. 성능 최적화 규칙
9. 관리자 페이지 규칙
10. 배포 및 운영 규칙
11. 수정 및 업데이트 규칙

---

1. 프로젝트 철학

이 프로젝트는 다음 세 가지 원칙을 최우선으로 삼습니다.

원칙 1 — 처음부터 완성형으로 설계한다 기능을 추가할 때마다 기존 코드를 전면 수정하는 일이 없도록, 처음부터 확장 가능한 구조로 설계합니다. “나중에 고치면 되지”라는 생각은 금물입니다.

원칙 2 — 단순하고 명확하게 작성한다 같은 기능이라면 코드가 짧고 읽기 쉬운 쪽을 선택합니다. 불필요한 코드, 사용하지 않는 변수, 중복된 로직은 즉시 제거합니다.

원칙 3 — 사용자가 많아져도 흔들리지 않는다 동시에 수백 명이 접속해도 느려지거나 오류가 나지 않도록, 성능과 안정성을 항상 고려하며 개발합니다.

---

2. 기술 스택 규칙

프로젝트에서 사용하는 기술은 아래 목록으로 고정합니다. 새로운 라이브러리를 추가할 때는 반드시 기존 기술로 해결이 불가능한지 먼저 검토해야 합니다.

영역	기술	선택 이유
프론트엔드 프레임워크	React 19 + TypeScript	컴포넌트 재사용, 타입 안전성
스타일링	Tailwind CSS 4	빠른 UI 작성, 일관된 디자인 토큰
UI 컴포넌트	shadcn/ui	접근성 보장, 커스터마이징 용이
백엔드 서버	Express.js (Node.js)	가볍고 빠른 API 서버
데이터베이스	PostgreSQL	안정성, 관계형 데이터 처리
ORM (DB 접근 도구)	Drizzle ORM	타입 안전, 가벼운 쿼리
인증	JWT (JSON Web Token)	무상태 인증, 확장성
파일 저장	CDN (manus-upload)	서버 부하 분산
라우팅	Wouter	가벼운 클라이언트 라우팅

> ***규칙:*** *위 목록에 없는 라이브러리를 추가하려면 반드시 이유를 문서에 기록하고 승인을 받아야 합니다.*

---

3. 폴더 및 파일 구조 규칙

```
joych-homepage/
├── client/                    # 방문자 홈페이지 (프론트엔드)
│   └── src/
│       ├── pages/             # 페이지 단위 컴포넌트
│       │   ├── Home.tsx       # 메인 홈페이지
│       │   └── admin/         # 관리자 페이지 (하위 폴더)
│       ├── components/        # 재사용 가능한 UI 컴포넌트
│       │   ├── ui/            # shadcn/ui 기본 컴포넌트
│       │   └── church/        # 교회 전용 컴포넌트
│       ├── hooks/             # 커스텀 훅 (데이터 불러오기 등)
│       ├── lib/               # 유틸리티 함수
│       │   ├── api.ts         # API 호출 함수 모음
│       │   └── auth.ts        # 인증 관련 함수
│       └── index.css          # 전역 스타일 (색상 변수 등)
├── server/                    # 백엔드 API 서버
│   ├── index.ts               # 서버 진입점
│   ├── routes/                # API 라우트 (기능별 분리)
│   │   ├── auth.ts            # 로그인/인증 API
│   │   ├── settings.ts        # 사이트 설정 API
│   │   ├── popups.ts          # 팝업 관리 API
│   │   ├── banners.ts         # 배너 관리 API
│   │   ├── menus.ts           # 메뉴 관리 API
│   │   ├── sermons.ts         # 설교 영상 API
│   │   ├── news.ts            # 교회 소식 API
│   │   ├── members.ts         # 새가족 등록 API
│   │   └── affiliates.ts      # 관련 기관 API
│   ├── middleware/            # 공통 처리 로직
│   │   ├── auth.ts            # 인증 확인 미들웨어
│   │   └── validate.ts        # 입력값 검증 미들웨어
│   └── db/                    # 데이터베이스 관련
│       ├── schema.ts          # DB 테이블 구조 정의
│       └── index.ts           # DB 연결 설정
├── shared/                    # 프론트엔드·백엔드 공통 타입
│   └── types.ts               # 공통 TypeScript 타입 정의
└── DEVELOPMENT_RULES.md       # 이 문서
```

파일 명명 규칙:

대상	규칙	예시
컴포넌트 파일	PascalCase (대문자 시작)	`HeroSection.tsx`
유틸리티/훅 파일	camelCase (소문자 시작)	`useSermons.ts`
API 라우트 파일	소문자 복수형	`sermons.ts`
상수	UPPER_SNAKE_CASE	`MAX_FILE_SIZE`

---

4. 코드 작성 규칙

4-1. 컴포넌트 작성 원칙

하나의 컴포넌트는 하나의 역할만 합니다. 컴포넌트가 100줄을 넘어가면 분리를 검토해야 합니다.

```
// ✅ 올바른 예: 역할이 명확하게 분리됨
<HeroSection />
<QuickMenuSection />
<SermonsSection />

// ❌ 잘못된 예: 하나의 컴포넌트에 모든 것을 넣음
<Home /> // 500줄짜리 컴포넌트
```

4-2. 데이터 불러오기 원칙

API 호출은 반드시 `hooks/` 폴더의 커스텀 훅으로 분리합니다. 페이지 컴포넌트 안에 직접 `fetch()`를 쓰지 않습니다.

```
// ✅ 올바른 예
const { sermons, isLoading } = useSermons();

// ❌ 잘못된 예
useEffect(() => {
  fetch('/api/sermons').then(...) // 페이지 안에 직접 API 호출
}, []);
```

4-3. 타입 정의 원칙

`any` 타입 사용을 금지합니다. 모든 데이터 구조는 `shared/types.ts`에 TypeScript 타입으로 정의합니다.

```
// ✅ 올바른 예
interface Sermon {
  id: number;
  title: string;
  youtubeUrl: string;
  date: string;
  category: 'sunday' | 'wednesday' | 'dawn';
}

// ❌ 잘못된 예
const sermon: any = { ... }
```

4-4. 주석 작성 원칙

코드 자체가 읽기 쉽게 작성되어야 하며, 주석은 "왜 이렇게 했는지"를 설명할 때만 사용합니다. "무엇을 하는지"를 설명하는 주석은 코드 리팩토링으로 대체합니다.

---

5. 데이터베이스 규칙

5-1. 테이블 설계 원칙

모든 테이블은 아래 공통 컬럼을 반드시 포함합니다.

```
id          SERIAL PRIMARY KEY,        -- 고유 번호 (자동 증가)
created_at  TIMESTAMP DEFAULT NOW(),   -- 생성 시각
updated_at  TIMESTAMP DEFAULT NOW()    -- 수정 시각
```

5-2. 컬럼 명명 규칙

규칙	예시
소문자 스네이크 케이스 사용	`youtube_url`, `is_active`
불리언(참/거짓) 컬럼은 `is_` 접두사	`is_active`, `is_deleted`
날짜 컬럼은 `_at` 또는 `_date` 접미사	`published_at`, `start_date`
외래키는 `_id` 접미사	`sermon_id`, `user_id`

5-3. 데이터 삭제 원칙

데이터를 실제로 삭제(DELETE)하지 않고, `is_deleted = true` 로 표시하는 **소프트 삭제** 방식을 사용합니다. 실수로 삭제해도 복구가 가능합니다.

```sql
-- ❌ 잘못된 예: 영구 삭제
DELETE FROM news WHERE id = 5;

-- ✅ 올바른 예: 소프트 삭제
UPDATE news SET is_deleted = true WHERE id = 5;
```

5-4. 인덱스 규칙

자주 조회하는 컬럼에는 인덱스를 설정하여 검색 속도를 높입니다.

```sql
-- 자주 사용하는 조회 조건에 인덱스 추가
CREATE INDEX idx_news_is_deleted ON news(is_deleted);
CREATE INDEX idx_sermons_category ON sermons(category);
```

---

6. API 설계 규칙

6-1. URL 구조 (REST 방식)

모든 API는 아래 패턴을 따릅니다.

동작	HTTP 메서드	URL 예시
목록 조회	GET	`/api/sermons`
단건 조회	GET	`/api/sermons/5`
새로 만들기	POST	`/api/sermons`
수정	PUT	`/api/sermons/5`
삭제	DELETE	`/api/sermons/5`

6-2. 응답 형식 통일

모든 API 응답은 아래 형식을 따릅니다.

```
// 성공 응답
{
  "success": true,
  "data": { ... }
}

// 실패 응답
{
  "success": false,
  "error": "오류 메시지"
}
```

6-3. API 버전 관리

향후 API 구조가 변경될 경우를 대비해 URL에 버전을 포함합니다.

```
/api/v1/sermons   ← 현재 버전
/api/v2/sermons   ← 미래에 구조가 바뀔 경우
```

6-4. 페이지네이션 (목록이 많을 때)

목록 API는 반드시 페이지 단위로 나눠서 응답합니다. 한 번에 모든 데이터를 불러오면 서버가 느려집니다.

```
GET /api/news?page=1&limit=10
```

---

7. 보안 규칙

7-1. 인증 체계

관리자 페이지의 모든 API는 JWT 토큰 인증을 통과해야 합니다. 토큰 없이 접근하면 `401 Unauthorized` 오류를 반환합니다.

```
요청 헤더: Authorization: Bearer {토큰값}
```

7-2. 비밀번호 규칙

비밀번호는 절대 평문(그대로)으로 저장하지 않습니다. 반드시 `bcrypt` 로 암호화하여 저장합니다.

```
// ✅ 올바른 예
const hashedPassword = await bcrypt.hash(password, 12);

// ❌ 잘못된 예
db.insert({ password: "1234" }); // 평문 저장 금지
```

7-3. 입력값 검증 규칙

모든 API 입력값은 서버에서 반드시 검증합니다. 프론트엔드 검증만으로는 부족합니다.

```
// 제목이 비어있거나 200자를 넘으면 거부
if (!title || title.length > 200) {
  return res.status(400).json({ success: false, error: "제목을 확인해주세요."
});
}
```

7-4. 파일 업로드 보안

이미지/영상 업로드 시 아래 제한을 반드시 적용합니다.

항목	제한
허용 이미지 형식	jpg, jpeg, png, webp
허용 영상 형식	mp4, webm
최대 이미지 용량	10MB
최대 영상 용량	500MB

7-5. 환경 변수 규칙

데이터베이스 비밀번호, JWT 비밀키 등 민감한 정보는 코드에 직접 쓰지 않고 환경 변수(`.env`)로 관리합니다.

```
// ✅ 올바른 예
const jwtSecret = process.env.JWT_SECRET;

// ❌ 잘못된 예
const jwtSecret = "my-secret-key-1234"; // 코드에 직접 작성 금지
```

---

8. 성능 최적화 규칙

8-1. 이미지/영상 최적화

홈페이지에 사용하는 모든 이미지와 영상은 CDN을 통해 제공합니다. 서버에 직접 저장하지 않습니다.

- 이미지: WebP 형식 우선 사용 (JPG 대비 30% 용량 절감)
- 영상: 히어로 배경 영상은 10MB 이하로 압축
- 이미지 지연 로딩(Lazy Loading) 적용: 화면에 보일 때만 불러옴

8-2. API 응답 캐싱

자주 바뀌지 않는 데이터(메뉴 구조, 관련 기관 목록 등)는 캐시를 적용하여 DB 조회 횟수를 줄입니다.

```
// 메뉴 데이터는 5분간 캐시
res.setHeader('Cache-Control', 'public, max-age=300');
```

8-3. 데이터베이스 연결 풀링

동시 접속자가 많을 때 DB 연결이 부족하지 않도록 연결 풀(Connection Pool)을 설정합니다.

```
// DB 연결을 최대 20개까지 동시에 유지
const pool = new Pool({ max: 20 });
```

8-4. 프론트엔드 번들 최적화

사용하지 않는 컴포넌트나 라이브러리가 최종 파일에 포함되지 않도록 코드 스플리팅(Code Splitting)을 적용합니다.

```
// 관리자 페이지는 방문자가 접속할 때 불러오지 않음 (필요할 때만 로드)
const AdminPage = lazy(() => import('./pages/admin/AdminDashboard'));
```

---

9. 관리자 페이지 규칙

9-1. 접근 제한

관리자 페이지(`/admin/*`)는 로그인한 관리자만 접근할 수 있습니다. 비로그인 상태에서 접근하면 로그인 페이지로 자동 이동합니다.

9-2. 권한 등급

등급	이름	할 수 있는 것
1등급	최고 관리자	모든 기능 + 관리자 계정 생성/삭제
2등급	일반 관리자	콘텐츠 관리 (소식, 영상, 팝업 등)
3등급	편집자	소식 작성/수정만 가능

9-3. 작업 로그

관리자가 중요한 작업(삭제, 설정 변경 등)을 수행하면 로그를 기록합니다. “누가, 언제, 무엇을 했는지” 추적이 가능해야 합니다.

9-4. UI 일관성

관리자 페이지의 모든 화면은 동일한 레이아웃(좌측 사이드바 + 우측 콘텐츠 영역)을 사용합니다. 새로운 관리 메뉴를 추가할 때 레이아웃을 새로 만들지 않고 기존 레이아웃을 재사용합니다.

---

10. 배포 및 운영 규칙

10-1. 배포 전 체크리스트

배포 전 아래 항목을 반드시 확인합니다.

- [ ] TypeScript 오류 없음 (`tsc --noEmit` 통과)
- [ ] 콘솔 오류 없음 (브라우저 개발자 도구 확인)
- [ ] 모바일 화면에서 레이아웃 깨짐 없음

- [ ] 관리자 로그인 동작 확인
- [ ] 환경 변수 설정 완료 (`.env` 파일)
- [ ] 이미지/영상 CDN URL 정상 동작 확인

10-2. 체크포인트 저장 규칙

아래 상황에서는 반드시 체크포인트(스냅샷)를 저장합니다.

- 새로운 기능 추가 완료 후
- 데이터베이스 구조 변경 전
- 대규모 코드 수정 전

10-3. 롤백 원칙

문제가 발생하면 즉시 이전 체크포인트로 롤백합니다. 오류가 난 상태에서 계속 수정하지 않습니다.

---

11. 수정 및 업데이트 규칙

11-1. 기능 추가 원칙

새로운 기능을 추가할 때는 아래 순서를 따릅니다.

```
1. DB 테이블/컬럼 추가 (schema.ts 수정)
2. 백엔드 API 라우트 추가 (routes/ 폴더)
3. 프론트엔드 훅 추가 (hooks/ 폴더)
4. UI 컴포넌트 추가 (components/ 폴더)
5. 페이지에 컴포넌트 연결
```

11-2. 기존 코드 수정 원칙

기존에 잘 동작하는 코드는 건드리지 않습니다. 수정이 필요하면 새로운 함수/컴포넌트를 만들어 교체하는 방식을 사용합니다.

11-3. 의존성(라이브러리) 업데이트 원칙

라이브러리 버전 업데이트는 한 번에 하나씩, 테스트 후 진행합니다. 한꺼번에 여러 라이브러리를 업데이트하면 어디서 오류가 났는지 찾기 어렵습니다.

변경 이력

날짜	버전	변경 내용
2026.04.05	v1.0	최초 작성

> *이 문서에 대한 질문이나 수정 요청은 담당자에게 문의하세요.* ***"처음부터 제대로, 나중에 편하게"***